

MANUAL DEL PROGRAMADOR

Inventario de Alimentos

Versión del documento: 1.0

Fecha de elaboración: dd/11/2026

Autor(es) / Equipo de desarrollo: Laura Marbella Bernal Justiniano

Maritza Rosibel Elías Carrillo

Josseline Sofia Miranda Miranda

Katherine Jazmín Ramirez Rivas

Ruth Abigail Salazar Benitez

Brandon Moíses Vasquez Alvarado

Curso / Asignatura: Modulo “C”

Control de Versiones

Version	Fecha	Autor	Descripción del Cambio
1.0	-11-2026	**Laura Marbella Bernal Justiniano** **Maritza Rosibel Elías Carrillo** **Josseline Sofia Miranda Miranda** **Katherine Jazmín Ramirez Rivas** **Ruth Abigail Salazar Benitez** **Brandon Moíses Vasquez Alvarado**	Versión inicial

INDICE

Contenido

1.INTRODUCCIÓN 5
2.OBJETIVO DEL MANUAL 6
2.1. Objetivo general 6
2.2. Objetivos específicos 6
2.3. Alcance del manual 7
3.ARQUITECTURA DEL SISTEMA 7
3.1. Tipo de arquitectura 8
3.2 Diagrama de arquitectura general 9
3.3. Descripción de capas del sistema 9
Capa de presentación 9
Capa de lógica de negocio 10
Capa de acceso a datos 12
Comunicación entre las capas 13
4.Tecnologías y Herramientas Utilizadas 14
5.ESTUCTURA DEL PROYECTO 14
DESCRIPCION DE CARPETAS Y ARCHIVOS PRINCIPALES 15
6. MODELO DE DATOS / BASE DE DATOS 16
Descripción de tablas/colecciones y sus campos 17
● Diccionario de datos 20
7. Instalación y Configuración del Entorno de Desarrollo 22
1. Requisitos previos 22
2. Clonación del repositorio 23
3. Instalación de dependencias 23
4. Configuración de la base de datos 24
5. Comandos para ejecutar el proyecto en modo desarrollo 24
6. Comandos para compilar / generar build de producción 24
8. Documentación de la Aplicación (Páginas y Scripts PHP) 25
8.1 Estructura de Páginas y Scripts 25
8.2 Comunicación Frontend – Backend 27

8.3 Conexión a la Base de Datos 28
9. Estándares y Convenciones de Codificación 31
Convención de nombres (variables, funciones, clases, archivos) 32
10. Módulos y Componentes Principales 33
11. Seguridad 35
12. Pruebas (Testing) 37
Matriz de pruebas funcionales propuesta 37
13. Despliegue (Deployment) 39
14. Mantenimiento y Control de Versiones 41
15. Anexos 43
Diagramas adicionales (casos de uso, secuencia, clases) 44
Diagrama de secuencia de consulta 44
Diagrama de secuencia del flujo de salida en el código 45

1.INTRODUCCIÓN

El presente manual técnico tiene como finalidad documentar los aspectos relacionados con el desarrollo, estructura, arquitectura y funcionamiento interno del Sistema de Control de Inventario de Alimentos de la institución. Este documento constituye una guía de referencia para los programadores y desarrolladores encargados de realizar actividades de mantenimiento, actualización, modificación y ampliación de las funcionalidades de la aplicación.

El sistema consiste en una aplicación web diseñada para facilitar la administración y el control de los productos alimenticios que pertenecen a la institución. Su desarrollo surge de la necesidad de contar con una herramienta informática que permita llevar un registro organizado de los alimentos disponibles, controlar las cantidades que ingresan y salen del inventario y mantener actualizada la información relacionada con las existencias de productos.

La aplicación permite gestionar los registros de los diferentes productos alimenticios, almacenar información relacionada con ellos y controlar los movimientos que se realizan dentro del inventario. De esta manera, se facilita el seguimiento de las entradas y salidas de alimentos, permitiendo identificar las cantidades disponibles y mantener un mejor control de los recursos de la institución.

Desde el punto de vista técnico, el sistema ha sido desarrollado utilizando tecnologías de programación web. Para la construcción de la interfaz se emplean HTML5 y CSS3, mientras que JavaScript permite incorporar funcionalidades

interactivas del lado del cliente. Por otra parte, PHP se utiliza como lenguaje de programación del lado del servidor, encargado de procesar las solicitudes y ejecutar las operaciones correspondientes. Para el almacenamiento y administración de la información se utiliza el sistema gestor de bases de datos MySQL.

Durante el proceso de desarrollo y pruebas se utiliza el entorno local XAMPP, que proporciona los servicios necesarios para ejecutar la aplicación mediante el servidor web Apache y administrar la base de datos. Asimismo, Visual Studio Code funciona como entorno de edición del código fuente.

Este manual está dirigido principalmente a programadores, desarrolladores y personal técnico que requiera comprender la implementación del sistema. Su contenido proporciona información sobre las tecnologías utilizadas, la arquitectura general, la comunicación entre los componentes y los procedimientos técnicos necesarios para facilitar futuras modificaciones y tareas de mantenimiento.

2.OBJETIVO DEL MANUAL

2.1. Objetivo general

Documentar de manera detallada los aspectos técnicos del desarrollo del Sistema de Control de Inventario de Alimentos de la institución, proporcionando una guía que permita a los programadores comprender su estructura, arquitectura, tecnologías, herramientas y funcionamiento interno, facilitando así las actividades de mantenimiento, actualización, corrección de errores y futuras ampliaciones del sistema.

2.2. Objetivos específicos

- Describir la estructura general y la arquitectura tecnológica utilizada durante el desarrollo de la aplicación web.
- Identificar las tecnologías, lenguajes de programación, herramientas y sistemas gestores de bases de datos empleados en la implementación del proyecto.
- Explicar la función que desempeña cada uno de los componentes que conforman el sistema.
- Documentar la comunicación entre la interfaz de usuario, el servidor web, la lógica de programación y la base de datos.
- Proporcionar información técnica que facilite la comprensión y modificación del código fuente por parte de otros desarrolladores.
- Establecer una referencia para realizar procedimientos de mantenimiento preventivo y correctivo.
- Facilitar la identificación y solución de posibles errores relacionados con el funcionamiento de la aplicación.

- Servir como documentación de apoyo para futuras actualizaciones, mejoras y nuevas funcionalidades.
- Describir los mecanismos utilizados para registrar, consultar y actualizar la información relacionada con los productos alimenticios y sus movimientos dentro del inventario.

2.3. Alcance del manual

El alcance de este manual comprende la documentación técnica de los principales elementos que intervienen en el desarrollo y funcionamiento del Sistema de Control de Inventario de Alimentos.

Se incluyen aspectos relacionados con la arquitectura del sistema, las tecnologías utilizadas, el entorno de desarrollo, la comunicación con la base de datos y la organización general de los componentes de la aplicación.

Asimismo, este documento está orientado a facilitar la continuidad del proyecto, permitiendo que futuros programadores puedan comprender la solución implementada sin necesidad de comenzar nuevamente el proceso de análisis y desarrollo.

3.ARQUITECTURA DEL SISTEMA

La arquitectura del Sistema de Control de Inventario de Alimentos describe la manera en que se encuentran organizados los diferentes componentes tecnológicos que permiten el funcionamiento de la aplicación. Esta estructura establece la comunicación entre la interfaz de usuario, el procesamiento de la información y el almacenamiento de los datos en la base de datos.

El sistema está diseñado para permitir la administración de los productos alimenticios de la institución, facilitando el registro de nuevos productos, el control de las entradas y salidas de alimentos, la actualización de las existencias y la consulta de la información almacenada.

Para su desarrollo se utilizan tecnologías web que trabajan de manera conjunta para garantizar el procesamiento y almacenamiento de la información. La arquitectura permite separar las funciones de presentación, lógica de negocio y acceso a datos, facilitando la organización del código fuente y el mantenimiento de la aplicación.

3.1. Tipo de arquitectura

El Sistema de Control de Inventario de Alimentos utiliza una arquitectura cliente-servidor, organizada mediante una separación lógica de tres capas: presentación, lógica de negocio y acceso a datos.

Este tipo de arquitectura permite distribuir las responsabilidades entre los diferentes componentes de la aplicación, facilitando la comunicación y el intercambio de información.

Arquitectura cliente-servidor:

La arquitectura cliente-servidor permite que el usuario interactúe con el sistema mediante un navegador web, mientras que el servidor se encarga de procesar las solicitudes y comunicarse con la base de datos.

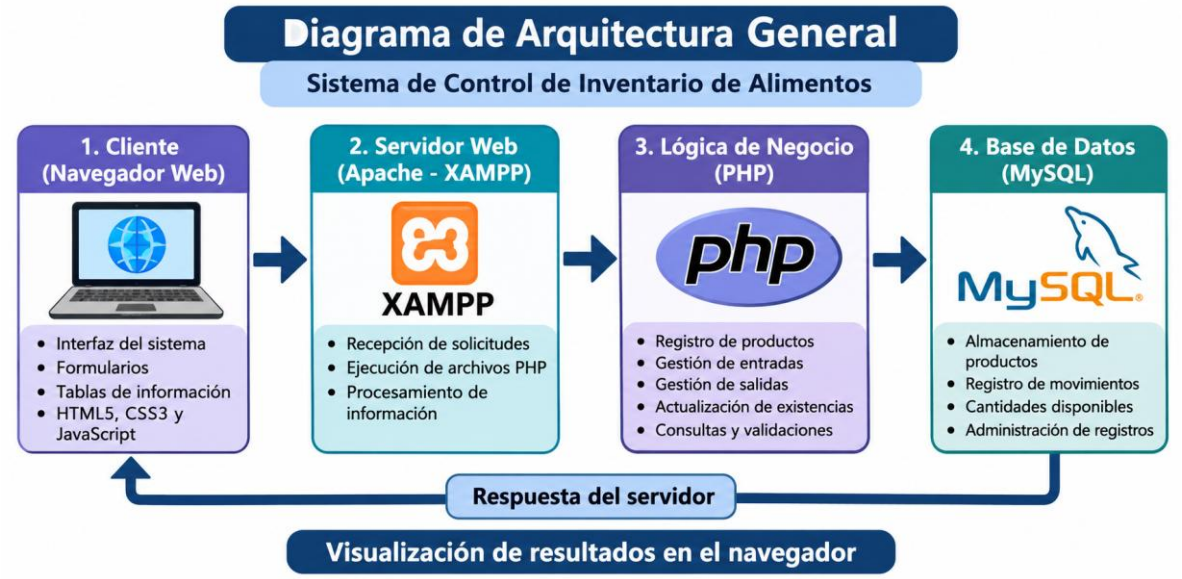
El cliente corresponde al navegador desde el cual se accede a la aplicación y se realizan las diferentes operaciones del inventario. El servidor utiliza Apache y PHP para recibir las solicitudes, procesar los datos y ejecutar las instrucciones necesarias en la base de datos MySQL.

Esta arquitectura permite centralizar el procesamiento de la información y mantener los registros del inventario almacenados en una base de datos.

Características de la arquitectura utilizada:

- Permite la interacción entre el navegador web y el servidor.
- Centraliza el procesamiento de las operaciones mediante PHP.
- Facilita el almacenamiento de información mediante MySQL.
- Permite separar la interfaz gráfica de los procesos internos.
- Facilita el mantenimiento y la actualización del código fuente.
- Permite administrar los registros de productos y sus movimientos desde una aplicación centralizada.

3.2 Diagrama de arquitectura general



3.3. Descripción de capas del sistema

El Sistema de Control de Inventario de Alimentos se encuentra organizado mediante una arquitectura de tres capas, las cuales permiten distribuir las diferentes responsabilidades que intervienen en el funcionamiento de la aplicación. Esta organización facilita la comunicación entre la interfaz gráfica, los procesos internos y la base de datos donde se almacena la información.

La separación de estas capas permite mantener una estructura organizada del sistema, facilitando la comprensión del código fuente, la identificación de errores y la realización de futuras modificaciones o actualizaciones.

Las tres capas principales que conforman el sistema son: capa de presentación, capa de lógica de negocio y capa de acceso a datos.

Capa de presentación

La capa de presentación corresponde a la interfaz gráfica mediante la cual el usuario interactúa con el Sistema de Control de Inventario de Alimentos. Su función principal es mostrar la información y proporcionar los elementos necesarios para realizar las diferentes operaciones disponibles en la aplicación.

Esta capa está desarrollada utilizando HTML5, CSS3 y JavaScript, tecnologías que permiten estructurar, diseñar y agregar interactividad a las diferentes páginas que conforman el sistema.

Mediante esta capa, el usuario puede acceder a las distintas secciones de la aplicación, visualizar los productos registrados, consultar las cantidades disponibles y utilizar los formularios correspondientes para administrar las entradas y salidas de alimentos.

Principales funciones de la capa de presentación:

- Mostrar la interfaz principal del sistema y sus diferentes módulos.
- Presentar los formularios utilizados para registrar y modificar productos.
- Visualizar las tablas con la información de los alimentos registrados.
- Mostrar los detalles y características de cada producto.
- Facilitar el acceso a los módulos de inventario, catálogo, entradas y salidas.
- Presentar las cantidades disponibles de los productos.
- Permitir la interacción mediante botones, menús, ventanas y formularios.
- Realizar validaciones básicas de los datos ingresados por el usuario.
- Mostrar mensajes de confirmación, advertencias y notificaciones sobre las operaciones realizadas.

Tecnologías utilizadas:

- HTML: permite definir la estructura de las páginas y los elementos de la interfaz.
- CSS: se encarga del diseño, distribución, colores y presentación visual.
- JavaScript: permite controlar eventos, realizar validaciones y agregar funcionalidades interactivas.

Esta capa tiene como finalidad proporcionar una interfaz organizada y facilitar la interacción entre el usuario y las funcionalidades del sistema.

Capa de lógica de negocio

La capa de lógica de negocio es la encargada de ejecutar los procesos internos que permiten el funcionamiento del Sistema de Control de Inventario de Alimentos. Su responsabilidad principal consiste en procesar la información recibida desde la interfaz y aplicar las operaciones necesarias de acuerdo con las funcionalidades implementadas.

Esta capa se desarrolla mediante PHP, lenguaje de programación que permite procesar las solicitudes, validar los datos y establecer la comunicación con la base de datos MySQL.

En esta parte del sistema se encuentran los procedimientos encargados de administrar los productos alimenticios, controlar los movimientos del inventario y actualizar la información cuando se realizan operaciones de entrada o salida de alimentos.

Principales funciones de la capa de lógica de negocio:

- Procesar la información enviada desde los formularios.
- Validar los datos correspondientes al registro de productos.
- Gestionar el registro de nuevos alimentos.
- Procesar las entradas de productos al inventario.
- Registrar las salidas de alimentos de acuerdo con las cantidades correspondientes.
- Realizar las operaciones necesarias para actualizar las existencias.
- Consultar la información de los productos registrados.
- Procesar las modificaciones de los datos de los productos.
- Controlar las operaciones de eliminación de registros cuando corresponda.
- Gestionar las solicitudes de información realizadas desde los diferentes módulos.
- Enviar los resultados del procesamiento hacia la interfaz para su visualización.

Por ejemplo, cuando se registra una entrada de alimentos, esta capa recibe los datos correspondientes al producto y la cantidad ingresada. Posteriormente, procesa la operación y realiza las instrucciones necesarias para guardar el movimiento y actualizar las existencias en la base de datos.

De igual manera, cuando se registra una salida, procesa la cantidad de alimentos que se retiran y ejecuta las operaciones correspondientes para mantener actualizado el inventario.

Esta capa es fundamental porque contiene los procedimientos que determinan cómo deben realizarse las operaciones del sistema y cómo debe administrarse la información de los productos alimenticios.

Tecnología utilizada:

- PHP: permite ejecutar los procesos internos, validar la información, gestionar las operaciones y comunicarse con la base de datos.

Capa de acceso a datos

La capa de acceso a datos es responsable de establecer la comunicación entre la aplicación y la base de datos MySQL. Su función principal consiste en almacenar, consultar, modificar y eliminar la información necesaria para el funcionamiento del sistema.

Esta capa permite conservar los registros relacionados con los productos alimenticios, las cantidades disponibles y los movimientos de entradas y salidas realizados dentro del inventario.

La comunicación se realiza mediante PHP, que ejecuta las consultas SQL necesarias para interactuar con las tablas de la base de datos.

Principales funciones de la capa de acceso a datos:

- Establecer la conexión entre la aplicación PHP y la base de datos MySQL.
- Almacenar los datos correspondientes a los productos registrados.
- Consultar la información de los alimentos existentes.
- Guardar los registros de entradas de productos.
- Almacenar los movimientos relacionados con las salidas de alimentos.
- Actualizar las cantidades disponibles cuando se realizan movimientos.
- Modificar la información almacenada cuando se actualizan los registros.
- Eliminar datos cuando la operación correspondiente se encuentra permitida.
- Recuperar los registros necesarios para mostrarlos en los diferentes módulos.
- Mantener organizada la información mediante las tablas definidas en la base de datos.

Por ejemplo, cuando el usuario consulta el catálogo de productos, esta capa permite recuperar los registros almacenados en MySQL para que posteriormente sean procesados y presentados en la interfaz.

Asimismo, cuando se registra una entrada o salida de alimentos, permite almacenar el movimiento correspondiente y actualizar los datos relacionados con las existencias del inventario.

Tecnologías utilizadas:

- MySQL: sistema gestor de bases de datos encargado de almacenar y administrar la información.
- SQL: lenguaje utilizado para ejecutar las consultas y operaciones sobre los registros.
- PHP: permite establecer la conexión y enviar las instrucciones necesarias a la base de datos.

Comunicación entre las capas

Las tres capas trabajan de manera integrada para garantizar el funcionamiento del Sistema de Control de Inventario de Alimentos.

La comunicación comienza cuando el usuario realiza una acción desde la interfaz gráfica. La capa de presentación recibe los datos y los envía para su procesamiento. Posteriormente, la capa de lógica de negocio ejecuta las operaciones correspondientes mediante PHP y, cuando es necesario, solicita información a la capa de acceso a datos.

La base de datos MySQL procesa las instrucciones recibidas y devuelve los resultados correspondientes. Finalmente, la aplicación procesa la respuesta y muestra la información actualizada en la interfaz del sistema.

Esta organización permite separar las responsabilidades de cada componente, facilita el mantenimiento del código fuente y contribuye a que las operaciones relacionadas con el inventario se realicen de manera organizada.

Ejemplo de comunicación entre las capas:

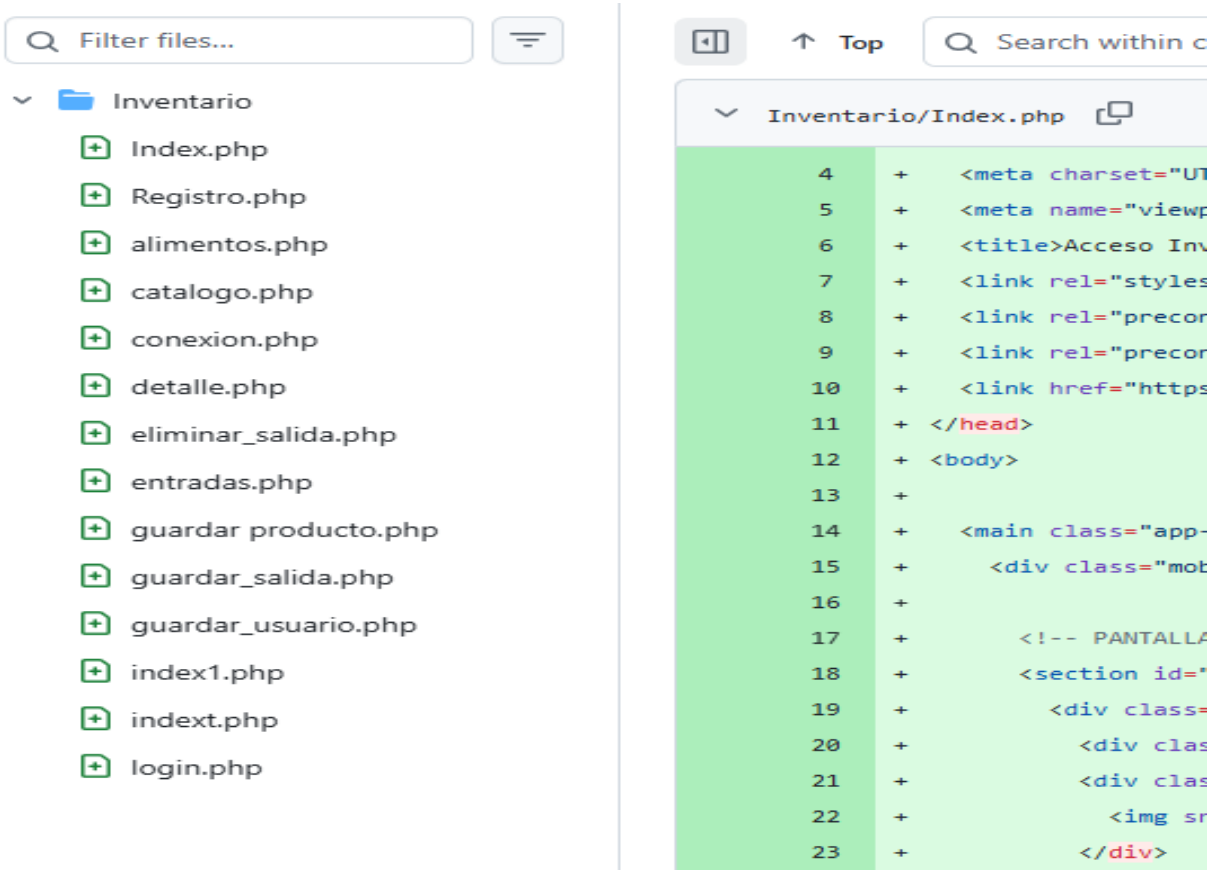
Cuando se registra una salida de alimentos:

1. **Capa de presentación:** el usuario ingresa el producto y la cantidad que desea registrar como salida.
2. **Capa de lógica de negocio:** PHP recibe los datos, realiza las validaciones correspondientes y procesa la operación.
3. **Capa de acceso a datos:** se ejecutan las consultas necesarias para guardar el movimiento y actualizar las existencias en MySQL.
4. **Respuesta del sistema:** la aplicación muestra el resultado de la operación y la información actualizada en la interfaz.

4.Tecnologías y Herramientas Utilizadas

CATEGORIA	TECNOLOGIA	VERSION
Lenguaje de marcado	HTML	5
Hojas de estilo	CSS	3
Lenguaje de programación (cliente)	JavaScript	6
Lenguaje de programación (servidor)	PHP	8
Base de datos	MySQL / MariaDB	8
Servidor web	Apache (XAMPP / WAMP / LAMP)	2.4
Control de versiones	Git	2.x
Otras librerías/dependencias		

5.ESTUCTURA DEL PROYECTO



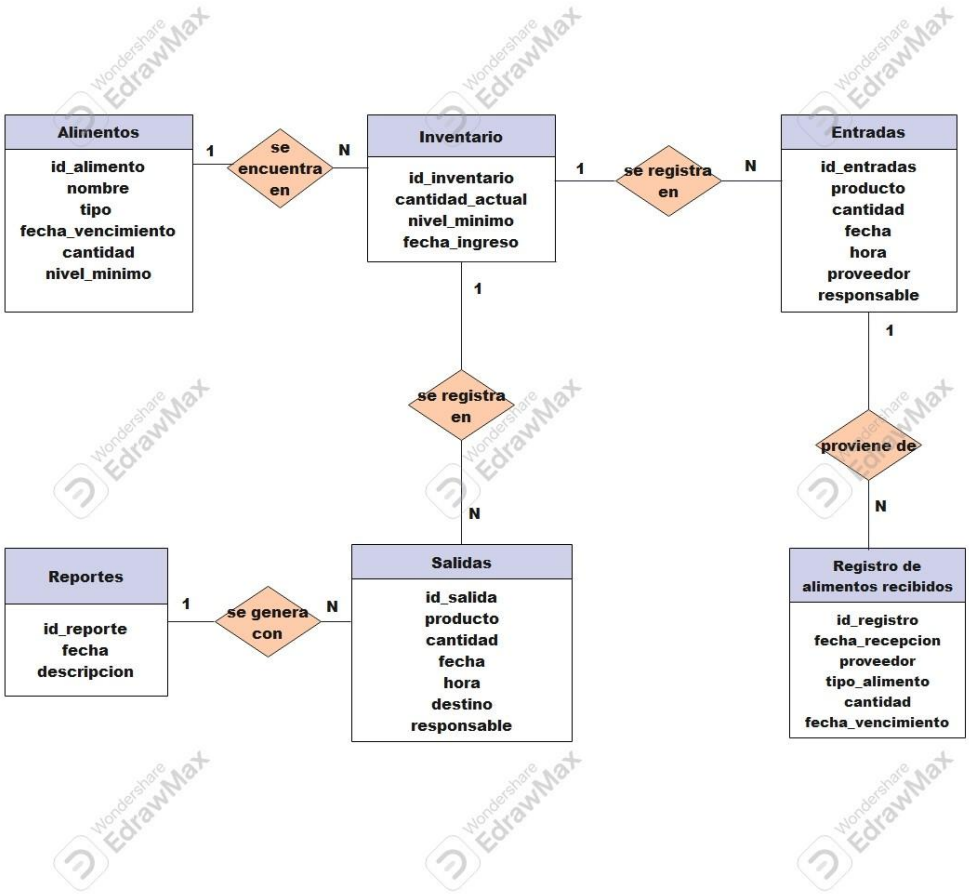
DESCRIPCION DE CARPETAS Y ARCHIVOS PRINCIPALES

CARPETA/ARCHIVO	DESCRIPCION
/img	Imágenes utilizadas por la interfaz
alimentos.php	Página para visualizar, administrar o registrar los alimentos del sistema.
catalogo.php	Muestra el catálogo general de productos, el código del producto, fecha de vencimiento.
conexion.php	Archivo centralizado que establece la conexión con la base de datos
detalle.php	Vista individual para mostrar los detalles específicos del registro de un producto
eliminar_salida.php	Script encargado de procesar y eliminar los registros de salidas del sistema.

entradas.php	Página para ver o registra un menú, seleccionar el menú correspondiente a ese día y calcula cuantas libras se utilizarán
Estilos.css / Estilos1.css / styles.css / input*.css / output.css	Hojas de estilo (CSS) que definen el diseño visual y la apariencia de la interfaz.
guardar_producto.php	Script que procesa los formularios para registrar y almacenar nuevos productos.
guardar_salida.php	Script que procesa el registro y almacenamiento de las salidas de productos.
guardar_usuario.php	Script encargado de procesar y guardar los datos de registro de nuevos usuarios.
Index.php	Index es utilizado para el registro de un usuario
index1.php	En esta página es de salida cuando se calcula en el menú cuantas libras saldrán de ese producto automáticamente se registra en esta página.
indext.php	Interfaz principal destinada al control y visualización de las entradas de productos.
Java.js / Java1.js / script.js	Scripts de JavaScript para validaciones, manipulación de la interfaz y llamadas asíncronas
login.php	Formulario e interfaz para el inicio de sesión y autenticación de usuarios.
package*. json	Archivos de configuración de dependencias y paquetes del proyecto (como npm).
Registro.php	Formulario de registro para la creación de cuentas de nuevos usuarios.

6. MODELO DE DATOS / BASE DE DATOS

- **Diagrama entidad-relación (Der)**



Descripción de tablas/colecciones y sus campos

CAMPO	DESCRIPCION
ID_Alimento	Identificador del producto
Nombre	Nombre del producto
Tipo	Categoría a la que pertenece el producto (granos, litros, etc.)
Fecha_vencimiento	Fecha en que vence el producto
Cantidad	Cantidad que se registra del producto
Nivel_mínimo	Cantidad mínima permitida para mantener disponible el alimento
ID_INVENTARIO	Identificador único del inventario

id_alimento	**Identificador del alimento relacionado con la tabla Alimentos**
cantidad_actual	**Cantidad actual de alimento disponible en el inventario**
nivel_minimo	**Cantidad mínima que debe mantenerse disponible**
fecha_ingreso	**Fecha en que el alimento ingresa al inventario**
ID_Reportes	**Identificador único de cada reporte**
id_inventario	**Identificador del inventario relacionado con el reporte**
Fecha	**Fecha en que se genera el reporte**
descripcion	**Información detallada del reporte generado**
ID_Registro	**Identificador único del registro de alimentos recibidos**
id_alimento	**Identificador del alimento relacionado con la tabla Alimentos**
fecha_recepción	**Fecha en que se reciben los alimentos**
proveedor	**Nombre del proveedor que entrega los alimentos**
tipo_alimento	**Nombre o categoría del alimento recibido**
cantidad	**Cantidad de alimentos recibidos**
fecha_vencimiento	**Fecha de vencimiento de los alimentos recibidos**
ID_ENTRADAS	**Identificador de entradas**
producto	**Nombre del producto**
cantidad	**Cantidad recibida**

fecha	**Fecha en que se registró el producto**
hora	**Hora en que se registró el producto**
Proveedor	**Nombre dl proveedor**
responsable	**Nombre de la persona que registro el producto**
ID_SALIDA	**Identificador de salida**
producto	**Nombre del producto que salió**
cantidad	**Cantidad que salió**
Fecha	**Fecha en que salió el producto**
hora	**Hora en que salió el producto**
destino	**Lugar al que se entregó el producto**
responsable	**Nombre de la persona que registro con la salida**

- **Relaciones entre entidades**

Entidades	Relación
Alimentos – Inventario	Cada alimento tiene un registro de inventario donde se controla su cantidad disponible y nivel mínimo.
Alimentos – Entradas	Un alimento puede registrar varias entradas al inventario, pero cada entrada corresponde a un solo alimento.

Alimentos – Salidas	De un mismo alimento se pueden realizar varias salidas según las necesidades.
Alimentos – Registro de alimentos recibidos	Un alimento puede tener varios registros de recepción, dependiendo de las veces que sea entregado.
Inventario – Reportes	De la información del inventario se pueden elaborar diferentes reportes para conocer su estado.

● Diccionario de datos

Nombre de la tabla	Campo	Tipo	Long	Nulo	descripción
Tabla: Alimentos	id_alimento	INT	_	No	Identificador unico del alimento
	nombre	VARCHAR	50	No	Nombre del alimento (arroz, leche, etc.)
	tipo	VARCHAR	30	No	Tipo de alimento
	fecha _ vencimiento	DATE	_	No	fecha de vencimiento del alimento
	cantidad	INT	_	No	cantidad disponible
Tabla: Inventario	Id_inventario	Entero	10	No	Indentificador del inventario
	Id_alimento	Entero	10	No	Relación con la tabla de alimentos
	cantidad_disponible	Entero	10	No	Cantidad actual en inventario
	nivel_mínimo	Entero	10	No	Nivel mínimo permitido
	fecha_ingreso	fecha	_	No	fecha de ingreso al inventario
Tabla.Reporte	id_reporte	Entero	10	No	identificador del reporte
	fecha	fecha	_	No	fecha del reporte
	descripción	texto	100	si	información del reporte diario
Tabla: Registro de alimentos recibido	id_registro	INT	10	No	Identificador unico del registro
	fecha_recepcion	DATE	_	No	fecha en que se reciben los alimentos
	proveedor	Varchar	100	No	Nombre del proveedor
	tipo_alimento	Varchar	100	No	Nombre del alimento
	Cantidad	INT	10	No	cantidad recibida
	fecha_vencimiento	DATE	_	No	Fecha de vencimiento del alimento
Tabla: Entradas	Id_Entrada	INT	10	No	Identificador único de entradas
	Producto	INT	10	No	Identificador del producto que ingresa
	Cantidad	DECIMAL	10,2	No	Cantidad del producto que ingresa

	Fecha	DATE	-	No	Fecha en que se realizó la entrada
	Hora	TIME	-	No	Hora en que se realiza la entrada
	Proveedor	VARCHAR	100	No	Nombre del proveedor que entrega el producto
	Responsable	VARCHAR	100	No	Nombre de la persona responsable de registrar la entrada
Tabla: Salidas	Id_salidas	INT	10	No	Identificador único de salidas
	Producto	INT	10	No	Identificador del producto que sale
	Cantidad	DECIMAL	10,2	No	Cantidad de producto que sale
	Fecha	DATE	-	No	Fecha en que se realiza la salida
	Hora	TIME	-	No	Hora en que se realiza la salida
	Destino	VARCHAR	100	No	Lugar o área a donde se envía el producto
	Responsable	VARCHAR	100	No	Nombre de la persona responsable de registrar la salida

7. Instalación y Configuración del Entorno de Desarrollo

1. Requisitos previos

Para desarrollar y ejecutar el sistema de Inventario de Alimentos se necesitan los siguientes programas y herramientas:

- **Visual Studio Code:** editor utilizado para escribir y modificar el código del proyecto.
- **HTML:** utilizado para crear la estructura de las páginas.
- **CSS:** utilizado para el diseño y apariencia del sistema.
- **JavaScript:** utilizado para agregar funciones e interactividad.
- **PHP:** utilizado para procesar los datos y realizar la comunicación con la base de datos.
- **MySQL:** sistema gestor de base de datos utilizado para almacenar la información.
- **XAMPP:** programa que proporciona el servidor Apache y MySQL para ejecutar el proyecto localmente.
- **Git y GitHub:** utilizados para guardar y compartir el código del proyecto.

2. Clonación del repositorio

Para obtener una copia del proyecto se utiliza Git, mediante el siguiente comando:

git clone https://github.com/Laura-Bernal12/Inventario-de-Alimentos-.git

Después de clonar el repositorio, se abre la carpeta del proyecto desde Visual Studio Code para trabajar con sus archivos.

3. Instalación de dependencias

El proyecto utiliza tecnologías como HTML, CSS, JavaScript y PHP, por lo que no requiere la instalación de paquetes adicionales mediante npm.

Para ejecutar el sistema es necesario instalar XAMPP, que incluye Apache y MySQL, los cuales permiten ejecutar los archivos PHP y administrar la base de datos.

4. Configuración de la base de datos

Para configurar la base de datos se realizan los siguientes pasos:

1. Instalar y abrir XAMPP.
2. Iniciar los servicios Apache y MySQL.
3. Ingresar a phpMyAdmin desde el navegador.
4. Crear una base de datos con el nombre inventario_alimentos.
5. Importar el archivo SQL que contiene la estructura y las tablas de la base de datos.
6. Configurar el archivo conexion.php con los datos correspondientes para establecer la conexión entre PHP y MySQL.
7. Comprobar que la conexión se realice correctamente.

5. Comandos para ejecutar el proyecto en modo desarrollo

Para ejecutar el proyecto en el entorno local se deben seguir estos pasos:

1. Abrir el panel de control de XAMPP.
2. Iniciar Apache y MySQL.
3. Colocar la carpeta del proyecto dentro de la carpeta htdocs de XAMPP.
4. Abrir el navegador web.
5. Ingresar a la siguiente dirección:

http://localhost/inventario/

De esta manera, se puede acceder al sistema y probar sus diferentes funciones desde el entorno local.

6. Comandos para compilar / generar build de producción

El proyecto está desarrollado utilizando HTML, CSS, JavaScript y PHP, por lo que no requiere un proceso de compilación mediante comandos especiales.

Para preparar el sistema para producción se deben realizar los siguientes pasos:

1. Verificar que los archivos HTML, CSS, JavaScript y PHP funcionen correctamente.
2. Comprobar la conexión con la base de datos.
3. Revisar que los formularios y las funciones del sistema trabajen correctamente.
4. Configurar los datos de conexión de la base de datos del servidor.
5. Copiar los archivos del proyecto al servidor web donde se alojará el sistema.
6. Importar la base de datos en el servidor de producción.

8. Documentación de la Aplicación (Páginas y Scripts PHP)

La aplicación está desarrollada como un sitio web dinámico, utilizando HTML, CSS, JavaScript y PHP y la conexión de la base de datos.

8.1 Estructura de Páginas y Scripts

Para cada página o script PHP:

Archivo	Descripción	Método	Parámetro recibido	Acción/salida
alimentos.php	Panel de control principal con estadísticas, stock y alertas.	GET / POST	$_GET['eliminar_id'] $_GET['msg'] $_POST['accion'] (como 'agregar_alimento_directo')	Muestra el resumen del inventario y permite agregar o borrar productos.
Catalogo.php	Lista completa y	GET / POST	eliminar_id, msg, accion.	Muestra los productos en

	filtrado de productos en inventario.			tabla, procesa altas y bajas de insumos.
conexión.php	Conexión central a la base de datos MySQL en XAMPP via PDO.	—	Credenciales BD (localhost, root, inventario_instituto)	Conecta los scripts con la base de datos o devuelve errores.
Detalle.php	Muestra la información detallada e individual de un producto seleccionado.	GET	$_GET['codigo'] $_GET['eliminar_id']	Consulta y despliega los datos completos del producto en la vista o procesa la eliminación si se solicita.
Eliminar_salida.php	Borrado asíncrono de un registro de salida.	POST (JSON)	file_get_contents('php://input') decodificado con el parámetro id.	Elimina el registro en la BD y responde con un JSON de confirmación.
Entradas.php	Menús escolares y cálculo automático de raciones por alumno.	GET / POST	$_POST (datos procesados mediante fetch/JSON o formularios)	Calcula las porciones necesarias de alimentos según los alumnos.
Guardar_producto.php	Descuento automático de stock por uso en menús.	POST (JSON)	JSON con alimentos, cantidades, fecha, hora y responsable	Resta el stock en catalogo e inserta el consumo en reportes de salida.
Guardar_salida.php	Registro transaccional de salidas y descuento de stock.	POST (JSON)	file_get_contents('php://input') decodificado con alimentos, responsable,	Actualiza el inventario en el catálogo y guarda el reporte de salida en la BD.
Guardar_usuario.php	Alta asíncrona de nuevos usuarios institucionale	POST (JSON)	file_get_contents('php://input') decodificado con nombre, email, password y rol.	Verifica duplicados de correo, encripta la

	s (vía JSON)..			contraseña de forma segura
Index.php	Pantalla principal de bienvenida, inicio de sesión y registro.	GET / POST	email, password, nombre, rol	Muestra la interfaz de acceso e inicia los formularios de Login y registro.
Index1.php	Reporte general de salidas de alimentos del inventario.	GET / POST	$_SESSION['usuario_id'] / $_SESSION['nombre'] (Validación de sesión).	Genera gráficas de salidas (Chart.js) y lista las descargas de alimentos.
Indext.php	Reporte general de entradas e ingresos de alimentos.	GET / POST	$_SESSION['usuario_id'] y $_SESSION['nombre'] (Validación de sesión).	Sincroniza los ingresos con el catálogo y muestra gráficas de control.
Registro.php	nuevas cuentas de usuario en el sistema.	POST	nombre, email, password, rol	Encripta la contraseña e inserta el nuevo usuario en la BD.
Login.php	Verificación de credenciales de inicio de sesión.	POST	$_POST['email'] (Correo electrónico ingresado).	Comprueba la contraseña encriptada e inicia la sesión del usuario.

8.2 Comunicación Frontend – Backend

La comunicación entre el Frontend y el Backend del sistema de Inventario de Alimentos se realiza mediante HTML, JavaScript y PHP. El Frontend permite al usuario ingresar y consultar información, mientras que el Backend se encarga de procesar los datos y comunicarse con la base de datos MySQL.

● Formularios HTML que envían datos a los scripts PHP mediante GET o POST

El sistema utiliza formularios HTML para recopilar información ingresada por los usuarios y enviarla a los archivos PHP para su procesamiento. Se utiliza principalmente el método POST para enviar datos de registro, inicio de sesión, entradas y salidas de alimentos.

● Uso de JavaScript (fetch / AJAX) para llamadas asíncronas a scripts PHP que retornan JSON

JavaScript permite realizar solicitudes a los archivos PHP mediante fetch o AJAX, sin necesidad de recargar toda la página. Los archivos PHP pueden devolver respuestas en formato JSON para indicar si una operación se realizó correctamente o si ocurrió algún error.

● Validaciones en el cliente y en el servidor

Se realizan validaciones para evitar que se envíen datos incorrectos o incompletos.

- **JavaScript:** comprueba que los campos estén completos y que la información ingresada tenga un formato correcto.
- **PHP:** verifica nuevamente los datos recibidos, realiza las validaciones necesarias y protege la información antes de guardarla en la base de datos.

● Manejo de sesiones con session_start () para autenticación y control de acceso entre páginas

PHP utiliza la función session_start () para iniciar y administrar las sesiones de los usuarios. Esto permite mantener la información de autenticación después de iniciar sesión y controlar el acceso a las diferentes páginas del sistema. Cuando el usuario cierra sesión, se finaliza la sesión correspondiente.

8.3 Conexión a la Base de Datos

La conexión a la base de datos del sistema de Inventario de Alimentos se realiza mediante PHP y MySQL. Esta conexión permite guardar, consultar, modificar y

eliminar la información de los usuarios y de los productos registrados en el inventario.

- **Motor de base de datos utilizado**
  El sistema utiliza MySQL como motor de base de datos para almacenar la información de los usuarios, productos, entradas y salidas de alimentos. Para administrar la base de datos se utiliza phpMyAdmin, incluido en XAMPP.

- **Archivo de conexión centralizado (conexion.php)**
  Se utiliza un archivo llamado conexion.php, encargado de establecer la conexión entre PHP y MySQL. Este archivo contiene los datos necesarios para conectarse a la base de datos, como el servidor, usuario, contraseña y nombre de la base de datos.

- **Uso de extensión mysqli o PDO para las consultas**

  Para realizar la conexión y ejecutar consultas en MySQL se puede utilizar la extensión mysqli de PHP. Esta permite trabajar con la base de datos mediante consultas para registrar, consultar, actualizar y eliminar información.

- **Uso de consultas preparadas (prepared statements)**

  Se utilizan consultas preparadas para realizar operaciones en la base de datos de una manera más segura. Estas permiten separar los datos ingresados por el usuario de las instrucciones SQL, ayudando a prevenir ataques de inyección SQL.

conexion.php > ...

```php
?php
servidor = "localhost";
base_datos = "inventario_instituto";
usuario = "root";
contrasena = "";

ry {
   // Conexión centralizada y segura utilizando PDO
   $conexion = new PDO("mysql:host=$servidor;dbname=$
   // Configurar PDO para que maneje excepciones ante
   $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::EF
   // Opcional: Descomenta la siguiente línea solo pa
   // echo "¡Conectado exitosamente!";
 catch (PDOException $e) {
   // Control de errores seguro y profesional
   error_log("Error de conexión: " . $e->getMessage()
   exit("No se pudo conectar a la base de datos del s
>
```

Incluir ejemplos de solicitud y respuesta (parámetros enviados y datos retornados), así como los mensajes o códigos de error controlados en cada script.

Ejemplos de solicitud y respuesta

Los archivos PHP reciben información enviada desde el sistema mediante solicitudes POST y devuelven respuestas indicando si la operación se realizó correctamente o si ocurrió algún error.

Inicio de sesión: recibe el correo electrónico y la contraseña. Como respuesta indica si los datos son correctos o incorrectos.

```
// Guardar la sesión activa
localStorage.setItem("cuentaActivaInventario", usuarioEncontrado.id);

alert(`✅ Sesión iniciada correctamente. Bienvenido/a, ${usuarioEncontrado.nombre
```

Registro de salidas: recibe el producto, cantidad, unidad y fecha de salida, y devuelve una respuesta indicando si la operación se realizó

correctamente.

```
{ nombre: "Frijoles", cantidad: 5.30, unidad: "lb" },
{ nombre: "Aceite", cantidad: 3.18, unidad: "L" },
{ nombre: "Bebida", cantidad: 212, unidad: "porciones" }
```

```
        };
        salidasInventario.push(nuevaSalida);
    });

    localStorage.setItem("salidasTableData", JSON.stringify(salidasI

    alert("✅ Registro guardado y enviado a Salidas correctamente.\r
}
```

9. Estándares y Convenciones de Codificación

Punto de la plantilla	Estado	Evidencia en los archivos
Convención de nombres	Documentado	Funciones JavaScript en camelCase y archivos PHP con nombres descriptivos: login.php, catalogo.php, entradas.php y conexion.php.

Formato de indentación y estilo	PROPUESTA DEFINIDA	Añadir .editorconfig, PHP_CodeSniffer (PSR-12) y ESLint; ejecutar validaciones antes de cada pull request. No están configurados en la versión actual.
Convenciones de commits	PROPUESTA DEFINIDA	Usar Conventional Commits (feat, fix, docs, test, chore), proteger main y trabajar con ramas feature/, fix/, docs/ y test/. La política se documenta aquí; no se observó configuración activa.
Comentarios y documentación	Documentado	Los comentarios existentes explican operaciones; el estándar recomendado agrega PHPDoc/JSDoc para funciones compartidas y mantiene ejemplos de uso junto a las validaciones clave.

Convención de nombres (variables, funciones, clases, archivos)

Las funciones JavaScript usan principalmente camelCase: invEntrada, invSalida, invConvertir, obtenerInventario, guardarEntrada, actualizarGraficas y handleLogin. En PHP se observan archivos descriptivos como login.php, Registro.php, alimentos.php, catalogo.php, detalle.php, entradas.php, indext.php, index1.php, conexion.php, guardar_salida.php y guardar_usuario.php. Las funciones comunes del inventario usan el prefijo inv y las constantes usan mayúsculas con guion bajo. En CSS aparecen clases descriptivas como menu-lateral, menu-admin y modal-contenido.

Formato de indentación y estilo (linter/formateador usado)

El código utiliza bloques con llaves, sangrías para anidar instrucciones y comentarios que separan grupos funcionales. PHP separa conexión, autenticación, catálogo, entradas, salidas y usuarios.

Convenciones de commits (ej. Conventional Commits)

El repositorio público de GitHub contiene la rama main y cuatro commits: creación del proyecto, unión del README, incorporación de documentación y actualización de la documentación. Estándar recomendado para continuar: proteger main; usar ramas feature/<tema>, fix/<tema>, docs/<tema> y test/<tema>; adoptar Conventional Commits (feat, fix, docs, test, chore) y exigir pull request con revisión y evidencia de pruebas. Esta política queda definida en este manual, pero no se afirma que esté configurada en GitHub.

Comentarios y documentación en el código

Los JavaScript contienen comentarios de sección y las páginas PHP incluyen comentarios sobre conexión PDO, validaciones, sesiones, consultas preparadas y respuestas JSON. Criterio propuesto: añadir PHPDoc a funciones PHP públicas y JSDoc a funciones

compartidas de JavaScript, con descripción, parámetros, valores de retorno y errores posibles. Para uniformar formato se recomienda .editorconfig, PHP_CodeSniffer con PSR-12 y ESLint; estas herramientas no están configuradas actualmente.

10. Módulos y Componentes Principales

La aplicación combina vistas PHP, JavaScript, CSS y una Base de Datos MySQL/MariaDB. Los módulos siguientes se derivan de las carpetas del sistema y de las tablas consultadas.

Módulo	Archivos principales	Estado
Cuentas y roles	Index.php, login.php, Registro.php, guardar_usuario.php	Implementado con sesión PHP y PDO
Catálogo e inventario	alimentos.php, catalogo.php, detalle.php, conexion.php	CRUD conectado a catalogo
Entradas	indext.php, entradas.php	Registra en registroentradas y actualiza catalogo
Salidas y consumo	index1.php, guardar_salida.php, guardar producto.php, eliminar_salida.php	Descuenta catalogo y registra reportes_salida_menu
Menús y gráficas	entradas.php, index1.php, Java1.js	Consulta datos y genera gráficas
Interfaz y estilos	Estilos.css, Estilos1.css, output.css, img/	Presentación y recursos visuales

Cuentas y roles

Punto del módulo	Información comprobada
Responsabilidad: qué hace este módulo dentro del sistema	Registra usuarios e inicia sesión mediante PHP. login.php consulta usuarios, verifica password_hash con password_verify y guarda usuario_id, nombre y rol en la sesión.
Archivos relacionados: rutas de los archivos principales	Index.php, login.php, Registro.php, guardar_usuario.php y conexion.php.
Funciones/clases clave: nombre y breve descripción	session_start, password_hash, password_verify, guardar_usuario.php y consultas preparadas sobre usuarios.
Dependencias: con qué otros módulos interactúa	Depende de PDO y de la tabla usuarios; las páginas protegidas redirigen a index.php si no existe usuario_id.

Catálogo e inventario

Punto del módulo	Información comprobada
Responsabilidad: qué hace este módulo dentro del sistema	Consulta, registra, edita y elimina productos; maneja código, categoría, cantidad, estado, ubicación y vencimiento.
Archivos relacionados: rutas de los archivos principales	alimentos.php, catalogo.php, detalle.php y conexion.php.

Funciones/clases clave: nombre y breve descripción	SELECT, INSERT, UPDATE y DELETE sobre catalogo; detalle.php consulta un producto por codigo.
Dependencias: con qué otros módulos interactúa	Depende de PDO y de la tabla catalogo; también relaciona registros con registroentradas.

Entradas

Punto del módulo	Información comprobada
Responsabilidad: qué hace este módulo dentro del sistema	Registra productos recibidos y guarda producto, cantidad, fecha, hora, proveedor y responsable. También actualiza el catálogo cuando corresponde.
Archivos relacionados: rutas de los archivos principales	indext.php y entradas.php.
Funciones/clases clave: nombre y breve descripción	INSERT y UPDATE en registroentradas y catalogo; consultas de entradas ordenadas por id.
Dependencias: con qué otros módulos interactúa	Depende de conexion.php, usuarios para el responsable y las tablas registroentradas y catalogo.

Salidas y consumo

Punto del módulo	Información comprobada
Responsabilidad: qué hace este módulo dentro del sistema	Valida la cantidad disponible, descuenta el producto y registra destino, responsable, fecha y hora.
Archivos relacionados: rutas de los archivos principales	index1.php, guardar_salida.php, guardar producto.php y eliminar_salida.php.
Funciones/clases clave: nombre y breve descripción	SELECT y UPDATE sobre catalogo; INSERT y DELETE sobre reportes_salida_menu; consultas agregadas para gráficas.
Dependencias: con qué otros módulos interactúa	Depende de PDO, sesión de usuario, catalogo y reportes_salida_menu.

Menús y gráficas

Punto del módulo	Información comprobada
Responsabilidad: qué hace este módulo dentro del sistema	Consulta menús, productos, salidas y cantidades agrupadas por producto o fecha para mostrar la información del inventario.
Archivos relacionados: rutas de los archivos principales	entradas.php, index1.php, Java1.js y recursos Chart cuando corresponda.
Funciones/clases clave: nombre y breve descripción	Consultas SELECT, SUM y GROUP BY sobre catalogo y reportes_salida_menu; funciones de gráficas en Java1.js.
Dependencias: con qué otros módulos interactúa	Depende de PDO, las tablas de inventario y los datos de la sesión.

Interfaz y estilos

Punto del módulo	Información comprobada
Responsabilidad: qué hace este módulo dentro del sistema	Presenta formularios, tablas, menús, alertas y recursos visuales del sistema.

Archivos relacionados: rutas de los archivos principales	Estilos.css, Estilos1.css, styles.css, input*.css, output*.css, img/ y las vistas PHP.
Funciones/clases clave: nombre y breve descripción	Clases CSS, formularios POST, tablas HTML y JavaScript de interacción.
Dependencias: con qué otros módulos interactúa	Depende de las vistas PHP, JavaScript, CSS y la información devuelta por PDO.

Flujo de datos actual

Paso	Qué ocurre en la carpeta principal del sistema
1. Captura	El usuario envía formularios POST desde las vistas PHP.
2. Sesión y validación	PHP inicia sesión, comprueba usuario_id y valida campos obligatorios.
3. Conexión	conexion.php crea una conexión PDO a inventario_instituto en localhost.
4. Consulta	Los scripts usan SELECT, INSERT, UPDATE y DELETE con parámetros preparados.
5. Respuesta	PHP redirige a la vista correspondiente o devuelve JSON en endpoints como guardar_salida.php.
6. Persistencia	Los cambios quedan en catalogo, registroentradas, reportes_salida_menu y usuarios.

Plan de normalización: conservar como nombres técnicos las tablas consultadas por PHP, actualizar el diccionario para mapear los nombres conceptuales y versionar el DDL validado. La matriz de cierre del apartado 15 establece el criterio de aceptación para el esquema y el despliegue.

11. Seguridad

Se conservan los cuatro puntos de la plantilla y se documenta lo que realmente aparece en PHP, JavaScript y las tablas de la Base de Datos.

Punto	Evidencia encontrada	Estado / medida de cierre
Autenticación	session_start, login.php, redirección si no hay usuario_id; la sesión conserva usuario_id, nombre y rol.	Implementado en el código; validar con pruebas de inicio/cierre y expiración de sesión.
Autorización por rol	Se guarda el rol en sesión, pero no se encontró una matriz formal de permisos por operación.	Propuesta: documentar rol × módulo × acción (leer, crear, editar, eliminar) y denegar por defecto toda acción no permitida.

Punto	Evidencia encontrada	Estado / medida de cierre
Contraseñas y datos sensibles	password_hash con PASSWORD_DEFAULT y password_verify; credenciales locales repetidas y errores visibles en guardar_usuario.php.	Parcial. Usar usuario de BD de privilegio mínimo y variables de entorno; nunca guardar claves en texto plano ni versionar secretos.
Validación e inyección SQL	trim, campos obligatorios y consultas preparadas observadas en PHP.	Mantener consultas preparadas; validar tipo, rango y unidad en servidor; probar entradas vacías, límites y payloads inesperados.
CSRF y sesiones	No se localizaron tokens CSRF en los formularios revisados.	Propuesta: token CSRF para todo POST que cambie estado; regenerar session ID al autenticar y cookies HttpOnly, Secure y SameSite en hosting HTTPS.
XSS y errores	guardar_usuario.php activa display_errors; las vistas imprimen datos que deben tratarse como no confiables.	Propuesta: htmlspecialchars al presentar texto, display_errors=Off en producción y error_log privado; al usuario solo mensaje genérico.

Mecanismo de autenticación y autorización

login.php consulta usuarios por email, verifica la contraseña con password_verify y guarda usuario_id, nombre y rol en $_SESSION. alimentos.php, catalogo.php, detalle.php, entradas.php, index1.php e indext.php llaman session_start y redirigen si falta usuario_id. El rol se conserva en la session.

Manejo de contraseñas y datos sensibles

Registro.php y guardar_usuario.php aplican password_hash con PASSWORD_DEFAULT; login.php usa password_verify. La configuración actual utiliza localhost, usuario root y contraseña vacía y muestra errores en guardar_usuario.php. Plan de cierre: crear un usuario de BD con privilegios mínimos, mover la configuración a variables de entorno, desactivar display_errors en producción y enviar detalles solo al registro del servidor. No publicar credenciales reales en el repositorio.

Validación de entradas

Los formularios PHP comprueban método POST, campos obligatorios y valores recibidos; los scripts usan trim, consultas preparadas y validaciones de existencia/cantidad. Las vistas JavaScript también contienen validaciones y confirmaciones. La profundidad de todas las validaciones del servidor debe probarse formalmente.

Protección contra vulnerabilidades comunes (inyección SQL, XSS, CSRF, etc.)

Los archivos PHP del sistema contienen sesiones, PDO y consultas preparadas. La matriz siguiente separa los controles observados de las medidas recomendadas: agregar token CSRF en cada operación que modifica datos, regenerar el ID de sesión al autenticar, codificar la salida HTML y devolver mensajes genéricos al usuario. Son medidas definidas

para la versión siguiente; el código no debe considerarse endurecido para producción hasta implementarlas y probarlas.

12. Pruebas (Testing)

Se ejecutaron comprobaciones estáticas, siete pruebas aisladas de funciones JavaScript y un smoke test HTTP. Los flujos que dependen de MySQL no se marcaron como aprobados: en los archivos disponibles no se encontró el esquema SQL y el entorno de verificación no tenía MySQL ni el controlador PDO MySQL. Los resultados y hallazgos verificables se separan de la matriz funcional propuesta.

Verificación ejecutada	Resultado	Evidencia / observación
Integridad del archivo fuente	APROBADA	La comprobación de integridad terminó sin errores.
Sintaxis PHP	APROBADA	php -l: 14 de 14 archivos PHP sin errores (PHP CLI 8.3.6).
Sintaxis JavaScript	APROBADA	node --check: 3 archivos externos y 5 bloques inline; 8 de 8 pasaron (Node.js v22.23.4).
Lógica aislada de inventario	APROBADA	7 de 7 pruebas sobre invConvertir e invAplicar: conversiones lb/g y ml/l, unidad incompatible, descuento, stock insuficiente y cantidad cero. Guion reproducible adjunto.
Rutas HTTP y mayúsculas	HALLAZGO CONFIRMADO	Servidor PHP local: /Index.php respondió 200 y /index.php respondió 404. Hay 17 referencias locales repetidas a index.php en seis páginas; el archivo disponible se llama Index.php.
Referencias locales	HALLAZGO CONFIRMADO	De 69 referencias locales revisadas, 17 no resuelven; todas son variantes en minúscula de index.php. Las demás referencias locales encontradas sí existen.
Script npm de pruebas	NO HAY SUITE	El script test declarado en package1.json se ejecutó en aislamiento y terminó con código 1: “Error: no test specified”.
Regla de stock en salida	HALLAZGO ESTÁTICO	invAplicar en JavaScript rechaza el exceso; guardar_salida.php calcula max(0, stock − consumo) y registra la salida. La ruta PHP no muestra rechazo explícito por exceder el stock; no se pudo probar con BD.
Tabla de reportes de salida	HALLAZGO ESTÁTICO	guardar producto.php inserta en “reportes de salida”; guardar_salida.php y eliminar_salida.php usan reportes_salida_menu. Debe confirmarse y unificarse contra el esquema real.
Integración CRUD con MySQL	NO EJECUTADA	No había servicio en el puerto 3306, no está instalado pdo_mysql y no se localizó un archivo SQL de esquema entre los archivos fuente; no se infiere que el CRUD funcione.

Matriz de pruebas funcionales propuesta

ID	Escenario	Resultado esperado / criterio	Estado
T01	Abrir entrada y navegar por las páginas.	La entrada y cada enlace local deben responder correctamente en el mismo sistema de mayúsculas/minúsculas.	Parcial ejecutada: /Index.php=200; /index.php=404. Corregir rutas antes de aceptar.
T02	Crear producto con datos válidos.	Debe guardarse una sola vez en catalogo y mostrarse con sus campos correctos.	No ejecutada: requiere MySQL y esquema.
T03	Enviar alta de producto con campos obligatorios vacíos.	Debe detenerse con validación clara y sin insertar un registro.	No ejecutada en navegador/BD.
T04	Registrar una entrada válida.	Debe guardar el movimiento y aumentar existencias una sola vez en la transacción esperada.	No ejecutada: requiere MySQL y esquema.
T05	Registrar una salida dentro del stock disponible.	Debe descontar la cantidad correcta y guardar el reporte en la tabla acordada.	No ejecutada: requiere MySQL; hay diferencia de nombres de tabla.
T06	Intentar salida mayor al stock o con unidad incompatible.	Criterio recomendado: rechazar la operación, informar el motivo y conservar el stock sin cambios.	No ejecutada con PHP/BD. Hallazgo: la lógica JS rechaza, pero el endpoint PHP usa max(0, …).
T07	Recargar la página y abrir sesión en otro navegador/dispositivo.	Los datos deben persistir en una fuente de datos única; validar expresamente si localStorage es solo caché local.	No ejecutada. Hay mezcla de datos de BD y localStorage; no se confirmó sincronización multiusuario.
T08	Acceder sin sesión y probar una cuenta autenticada por rol.	Sin sesión, debe bloquear/redirigir; con sesión válida, conservar usuario_id, nombre y rol y aplicar permisos documentados.	No ejecutada: requiere BD. El código guarda variables de sesión; permisos efectivos por rol no se verificaron. Consultar la matriz de seguridad del apartado 11.
T09	Crear y luego eliminar una salida.	Alta y eliminación deben operar sobre la misma tabla; la regla de reposición de stock debe quedar definida.	No ejecutada: discrepancia estática entre “reportes de salida” y reportes_salida_menu.
T10	Registrar usuario duplicado y verificar almacenamiento de clave.	Debe rechazar email duplicado y guardar solo hash de contraseña; nunca la clave en texto plano.	No ejecutada con BD; la revisión estática encontró password_hash/password_verify.

Tipo de pruebas implementadas (unitarias, integración, end-to-end)

Se ejecutaron siete pruebas aisladas sobre invConvertir e invAplicar del archivo JavaScript, además de verificaciones de sintaxis PHP/JavaScript y una prueba HTTP de ruta. No se ejecutaron pruebas de integración con base de datos ni pruebas end-to-end de los flujos de usuario.

Framework de pruebas utilizado

No se encontró una suite ni framework configurado. Se usaron PHP CLI 8.3.6 para lint y Node.js v22.23.4 para sintaxis/pruebas aisladas; estas herramientas no constituyen un framework de pruebas del proyecto. El script test de package1.json se ejecutó y terminó con código 1 mostrando “Error: no test specified”.

Cómo ejecutar las pruebas

Comprobaciones repetibles: ejecutar php -l en cada archivo PHP; node --check en Java.js, Java1.js y script.js; y node pruebas_inventario_helpers.js /ruta/Inventario/script.js para repetir las siete pruebas unitarias aisladas. Para el smoke test, iniciar php -S 127.0.0.1:8000 -t /ruta/Inventario y solicitar /Index.php y /index.php. Las pruebas T02–T10 requieren antes instalar PDO MySQL, levantar MySQL/MariaDB y contar con un esquema de prueba reproducible.

Cobertura de código esperada

Meta propuesta, no alcanzar al menos 80 % de cobertura de líneas en las funciones de inventario y cubrir el 100 % de los casos críticos de autenticación, unidades incompatibles, cantidad cero y salida superior al stock.

13. Despliegue (Deployment)

Se conservan los cinco puntos de la plantilla y se describen los archivos PHP, las carpetas de recursos y la configuración realmente entregada.

Punto	Estado	Evidencia / plan de cierre
Entornos existentes	PROPUESTA DEFINIDA	Separar desarrollo local, inventario_instituto_test y producción; usar credenciales y datos distintos. Actualmente el código apunta a localhost y no hay entornos separados.
Pasos de despliegue	SMOKE TEST PARCIAL	Verificado: /Index.php=200 y /index.php=404. Para completar el despliegue, instalar PDO MySQL, cargar esquema aprobado, corregir rutas y ejecutar T02–T10.
Servidor / hosting	RECOMENDACIÓN	Demostración: XAMPP local con Apache/PHP 8.3.x y MariaDB. Publicación: hosting PHP + PDO MySQL, HTTPS, copias de seguridad y acceso restringido a la BD.
Variables por entorno	PROPUESTA DEFINIDA	Externalizar DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, APP_ENV y APP_DEBUG; mantener secretos fuera de Git y de la carpeta pública.
CI/CD	PROPUESTA DEFINIDA	GitHub Actions en pull_request y push a main: PHP lint, Node syntax, tests JS; añadir integración MySQL después de versionar el esquema.

Desarrollo local: pasos recomendados

Paso	Acción
1	Instalar Node.js y npm/pnpm si se requiere regenerar estilos.
2	Desde la carpeta de la aplicación, iniciar php -S 127.0.0.1:8000 -t .; el servidor local permite servir las páginas PHP. Para módulos CRUD también se requiere MySQL/PDO MySQL.
3	Abrir /Index.php (smoke test HTTP 200) y revisar recursos; corregir antes los enlaces /index.php, que en servidor sensible a mayúsculas responden 404.
4	Regenerar CSS si se modifican clases de Tailwind.
5	Limpiar localStorage antes de una prueba independiente y registrar la evidencia.

Producción y configuración

Elemento	Estado
Aplicación PHP	Requiere servidor PHP y MySQL/MariaDB; conexion.php usa localhost e inventario_instituto.
Hosting	Propuesta de demostración: XAMPP local con Apache/PHP 8.3.x, PDO MySQL y MariaDB. No se declara un hosting productivo; para público se requiere HTTPS, BD aislada y respaldos.
Base de Datos	El código consulta catalogo, registroentradas, reportes_salida_menu y usuarios. Plan: versionar un DDL validado, crear inventario_instituto_test y comprobar altas, entradas, salidas, eliminación y rollback. CRUD no ejecutado en esta revisión; el endpoint guardar producto.php usa además “reportes de salida”, que debe unificarse.
Variables de configuración	Criterio: conexion.php leerá DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD del entorno; .env.example contendrá solo nombres y valores no secretos; .env real queda fuera de Git.
Credenciales y errores	Antes de producción: eliminar root/contraseña vacía, otorgar solo permisos requeridos, apagar display_errors, registrar fallos en log privado y devolver al usuario errores genéricos.

Entornos existentes (desarrollo, pruebas, producción)

Entornos recomendados: desarrollo local con PHP 8.3.x, Apache, PDO MySQL y MariaDB; pruebas con una base aislada inventario_instituto_test y datos ficticios; producción con host separado, HTTPS, usuario de BD restringido y respaldos.

Pasos para desplegar la aplicación

- Instalar PHP y habilitar la extensión PDO MySQL.
- Instalar MySQL/MariaDB y cargar en una base limpia el esquema SQL aprobado.
- Crear un usuario de aplicación con privilegios mínimos; evitar root y contraseña vacía.
- Configurar DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD desde el entorno; no publicar secretos.
- Corregir las referencias /index.php y unificar la tabla de reportes antes de conectar los módulos.
- Iniciar Apache/PHP y ejecutar T01–T10 primero en la base de pruebas; verificar las operaciones y el rollback.

Verificación realizada: /Index.php respondió HTTP 200 y /index.php respondió HTTP 404 en el servidor local.

Servidor / servicio de hosting utilizado

Servidor recomendado para una entrega académica de demostración: XAMPP local con Apache, PHP 8.3.x, PDO MySQL y MariaDB. Para publicación pública, elegir un hosting que permita esas tecnologías, HTTPS, acceso privado a la base y respaldos automatizables. No se identificó un proveedor productivo utilizado por el proyecto; el servidor PHP integrado se usó solo para smoke tests.

Variables de configuración por entorno

Configuración propuesta: cargar DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, APP_ENV y APP_DEBUG desde el entorno o un archivo local fuera de la carpeta pública. Incluir únicamente un .env.example sin secretos, excluir .env del control de versiones y adaptar conexion.php para leer esas variables. La versión actual conserva valores locales hard-coded y aún requiere ese cambio de código.

Proceso de CI/CD (si existe)

Flujo CI/CD propuesto: crear .github/workflows/ci.yml para ejecutar en cada pull request y push a main; pasos: lint de los 14 PHP, node --check de JavaScript, siete pruebas de helpers y, cuando exista un esquema reproducible, pruebas de integración contra un servicio MySQL aislado. Bloquear el merge si una etapa falla.

14. Mantenimiento y Control de Versiones

Se conservan los cuatro puntos originales y se describen las evidencias de mantenimiento presentes.

Punto	Estado	Evidencia / plan de trabajo
Flujo de trabajo con Git	Documentado	Repositorio público con rama main e historial de cuatro commits.

Punto	Estado	Evidencia / plan de trabajo
Nomenclatura de ramas	PROPUESTA DEFINIDA	feature/<tema>, fix/<tema>, docs/<tema> y test/<tema>; conservar main como rama estable. Aún no se observó aplicada en GitHub.
Revisión de código	PROCESO PROPUESTO	Pull request con issue, resumen, lista de cambios, evidencia de pruebas, migración/variables afectadas y revisión de seguridad; merge solo con aprobación y CI verde.
Incidencias / bugs	HALLAZGOS DOCUMENTADOS	Registrar y dar seguimiento a rutas index.php/Index.php, diferencias de regla de stock y nombres de tabla de salidas; cerrar cada issue después de corregir y repetir la prueba.

Flujo sugerido de mantenimiento

Paso	Flujo recomendado para cada cambio
1	Actualizar main y crear una rama feature/, fix/, docs/ o test/ con un solo objetivo.
2	Realizar commits pequeños en formato Conventional Commits: feat:, fix:, docs:, test: o chore:.
3	Ejecutar PHP lint, node --check y el guion de siete pruebas; adjuntar resultados. Para cambios de BD, usar la base de pruebas y comprobar rollback.
4	Abrir pull request con issue asociado, descripción, capturas si aplica, migración/configuración y checklist de seguridad. Solicitar revisión.
5	Integrar solo con aprobación y CI verde; etiquetar versiones estables y actualizar CHANGELOG, manual y procedimiento de respaldo.

No se localizaron pull requests en el repositorio revisado así que lo puse como debe de ser según investigue

Flujo de trabajo con Git (Gitflow, trunk-based, etc.)

El repositorio de GitHub usa la rama main y registra cuatro commits. Estrategia recomendada: main contiene entregas estables; cada cambio se desarrolla en una rama corta feature/<tema>, fix/<tema>, docs/<tema> o test/<tema>, y vuelve a main únicamente mediante pull request revisado y con comprobaciones exitosas.

Nomenclatura de ramas

Nomenclatura propuesta: feature/catalogo, feature/entradas, fix/salidas-stock, fix/rutas-mayusculas, docs/manual y test/inventario. Usar minúsculas y guiones; eliminar ramas al integrarlas. En el repositorio revisado solo se observó main.

Proceso de revisión de código (pull requests)

Checklist recomendado para cada pull request: objetivo y issue asociado; archivos/módulos modificados; pruebas ejecutadas y resultados; cambios de esquema o configuración; revisión de seguridad; confirmación de que no incluye credenciales. Solicitar al menos una revisión, exigir CI verde y registrar decisión de merge. No se observaron pull requests previos ni esta configuración activa.

Gestión de incidencias / bugs

Las funciones tomarRespaldo, restaurarRespaldo y restaurarClave recuperan datos locales, pero no sustituyen un registro de bugs. En esta revisión quedaron hallazgos reproducibles para registrar: referencias a index.php que devuelven 404 cuando el archivo es Index.php, divergencia entre la regla de stock del JavaScript y el endpoint PHP, y nombres distintos para la tabla de reportes de salida. Se recomienda abrir un issue por hallazgo, asignar responsable y no cerrar hasta repetir la prueba tras la corrección.

15. Anexos

Se conservan los puntos de la plantilla y se integran los archivos PHP, HTML, CSS, JavaScript, el informe SQL y el Diccionario de Datos del sistema.

Diagramas adicionales (casos de uso, secuencia, clases)

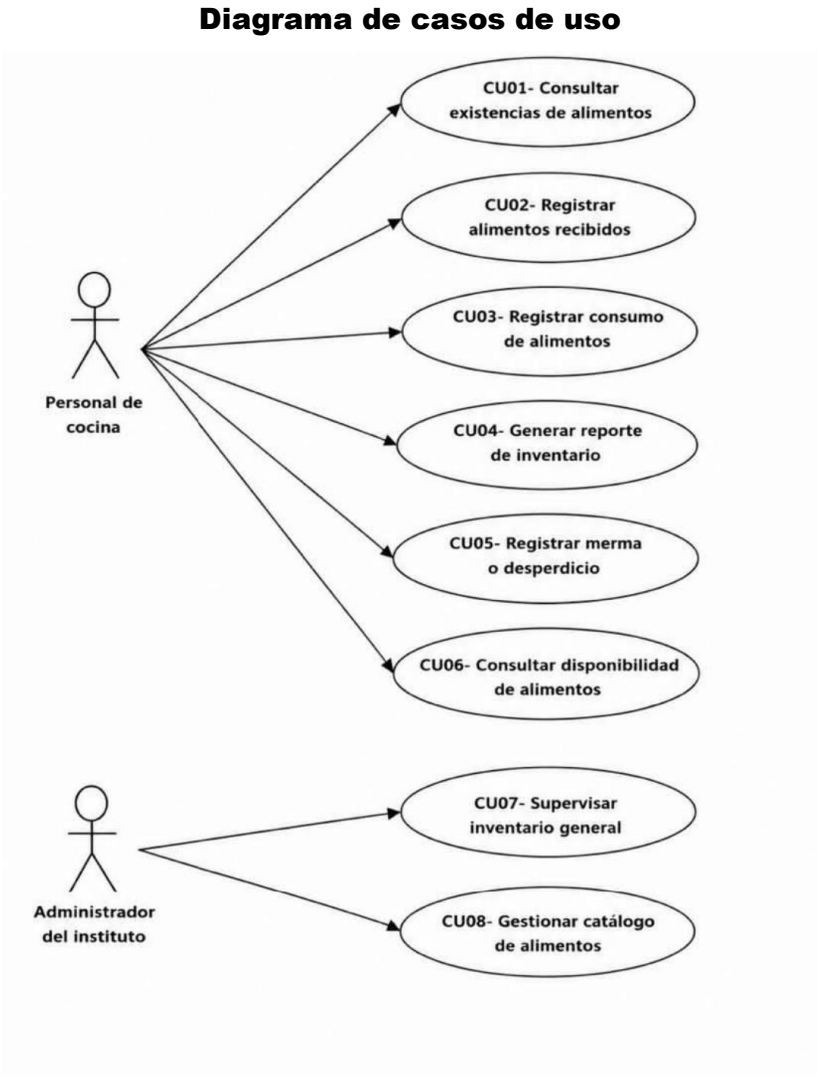


Figura 1. Diagrama de casos de uso. Fuente: "INSTITUTO NACIONAL PROFESORA BERTHA FIDELIA CAÑAS).pdf", página 8.

Diagrama de secuencia de consulta

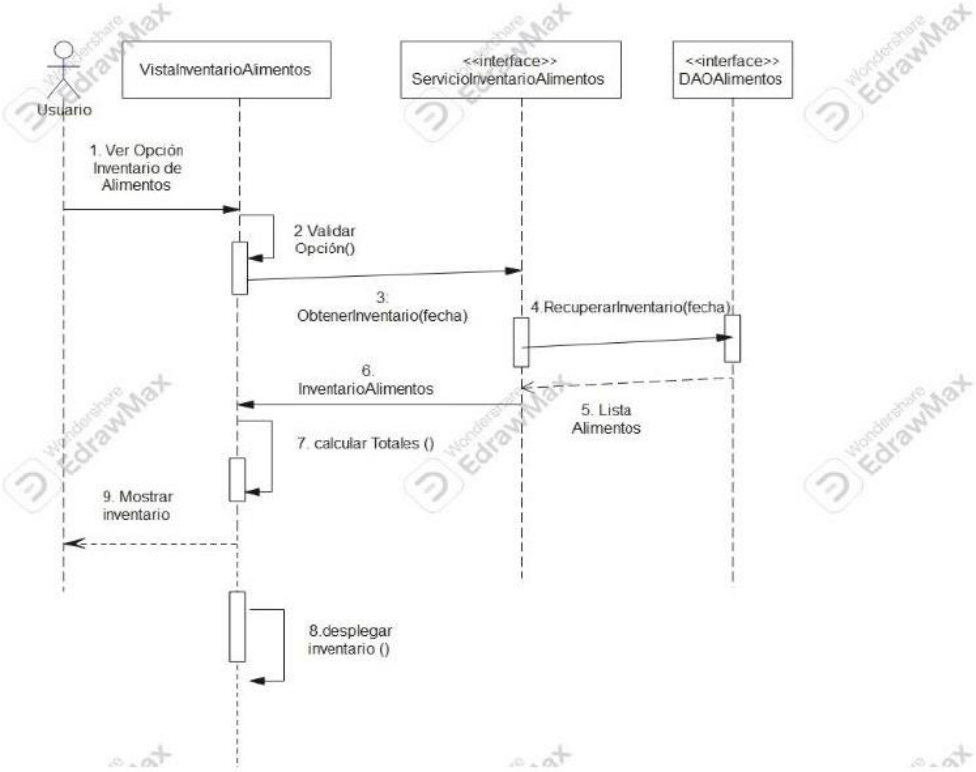


Figura 2. Secuencia de consulta del inventario. Fuente: "INSTITUTO NACIONAL PROFESORA BERTHA FIDELIA CAÑAS).pdf", página 21.

Diagrama de secuencia del flujo de salida en el código

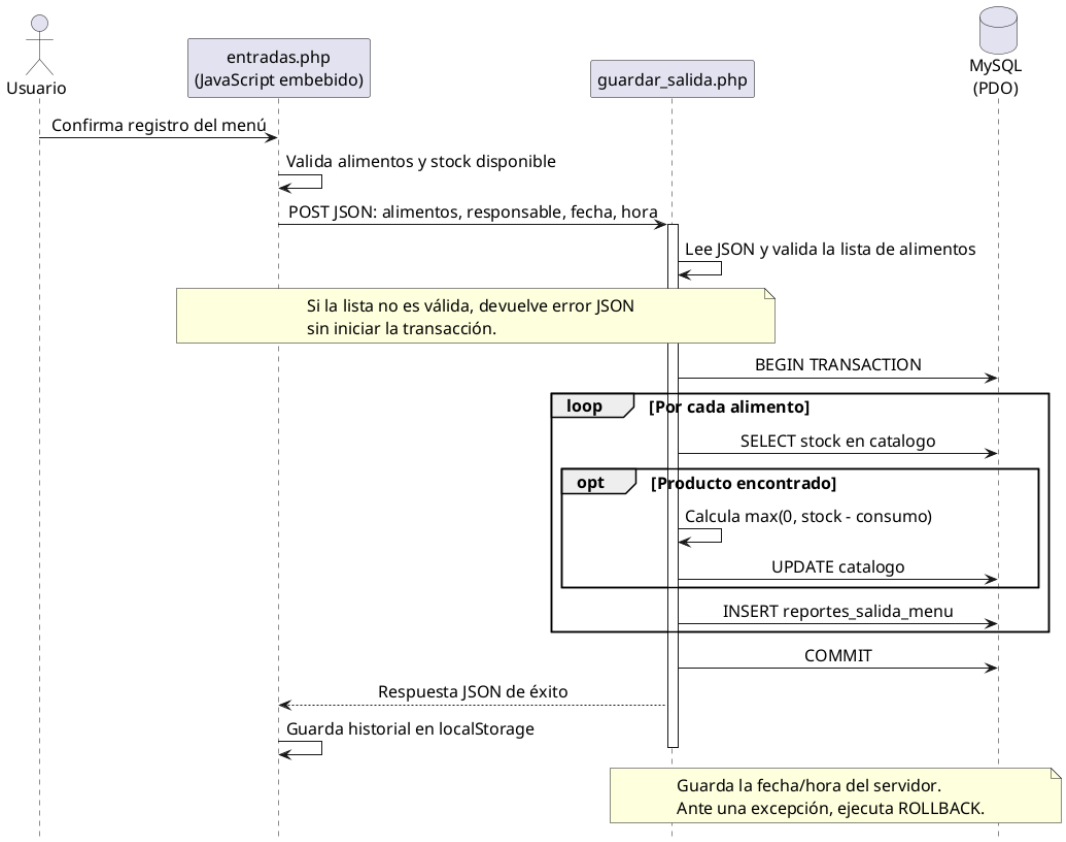


Figura 3. Secuencia derivada de entradas.php y guardar_salida.php.

Representación tipo clases basada en campos comprobados



Figura 4. Representación estructural tipo clases elaborada a partir de campos presentes en el código.

Glosario técnico

- **localStorage**
  Almacenamiento del navegador usado para productos, entradas, salidas, cuentas, preferencias de gráficas y respaldos.
- **Entrada**
  Movimiento que suma cantidad mediante invEntrada y se registra en entradasEscolar.

- **Salida**

  Movimiento que descuenta mediante invSalida o invAplicarMenu y se relaciona con salidasTableData.

- **Unidad**

  Texto como lb, kg, g, l, ml o unidades, normalizado y convertido por invConvertir cuando es compatible.

- **Tabla SQL**

  La aplicación PHP consulta catalogo, registroentradas, reportes_salida_menu y usuarios mediante PDO. El script DDL citado por la documentación debe adjuntarse y validarse con una importación limpia; no se ejecutó contra una base disponible en esta revisión.

Documentos utilizados

· **"INSTITUTO NACIONAL PROFESORA BERTHA FIDELIA CAÑAS).pdf"** — Documento de visión y alcance; contiene las figuras de casos de uso y secuencia citadas en este apartado.

· **"INSTITUTO NACIONAL PROFESORA BERTHA FIDELIA CAÑASS.docx 2 (1).pdf"** — Perfil del proyecto.

· **inventario_instituto (1).sql** — Definición y consultas de la base de datos.

· **Diccionario de Datos.xlsx** — Diccionario de datos y nombres conceptuales de entidades/tablas.

· **Archivos PHP** — alimentos.php, catalogo.php, conexion.php, detalle.php, eliminar_salida.php, entradas.php, guardar producto.php, guardar_salida.php, guardar_usuario.php, Index.php, index1.php, indext.php, login.php y Registro.php.

· **Archivos HTML, JavaScript y CSS** — Index.html, Index1.html, alimentos.html, catalogo.html, detalle.html, entradas.html, indext.html; Java.js, Java1.js, script.js; Estilos.css, Estilos1.css, input.css, input1.css, input2.css, input3.css, output.css, styles.css y hojas src/output.css, src1/output1.css, src2/output2.css y src3/output3.css.

· **Configuración y repositorio** — package.json, package1.json, package2.json, package3.json y sus lockfiles; repositorio público Laura-Bernal12/Inventario-de-Alimentos-, rama main e historial de cambios.

Archivos y Base de Datos entregados

Elemento	Contenido comprobado
Páginas HTML/PHP	Index.php, Registro.php, alimentos.php, catalogo.php, detalle.php, entradas.php, index1.php e indext.php; también existen vistas HTML.
JavaScript	script.js, Java.js y Java1.js.
CSS	Estilos.css, Estilos1.css, styles.css, input*.css, output*.css y variantes src*/output*.css.
Base de Datos	El SQL entregado define catalogo, registroentradas, reportes_salida_menu y usuarios; el PHP usa esas tablas.
Diccionario	Diccionario de Datos.xlsx con flujo y tablas conceptuales Alimentos, Inventario, Reporte y Registro de alimentos recibido.

Imágenes	img/instituto.jpg, img/logo-instituto.jpg e img/logo1.png.
PHP y configuración	conexion.php, login.php, Registro.php, guardar_usuario.php, guardar_salida.php, eliminar_salida.php y scripts de catálogo, entradas y salidas.